1. How to use VS Code for these Python topics

Step 1: Open VS Code

Open **Visual Studio Code**.

Step 2: Create a folder

Create a folder like:

Python_OOP_Intermediate

Open this folder in VS Code.

Step 3: Create Python files

Create files like:

- `classes_demo.py`
- `inheritance_demo.py`
- `iterators_demo.py`
- `decorators_demo.py`
- `json_regex_demo.py`

Or keep everything in one file:

- `main.py`

Step 4: Select Python interpreter

In VS Code:

- Press `Ctrl + Shift + P`
- Type `Python: Select Interpreter`
- Choose your installed Python version

Step 5: Run code

You can run Python code in VS Code by:

- clicking **Run** button
- or right-click → **Run Python File in Terminal**
- or open terminal and run:

python main.py

If `python` does not work, use:

py main.py

---

2. Object-Oriented Programming (OOP)

OOP is a programming style where we organize code using **classes** and **objects**.

Real-life idea:

- **Class** = blueprint
- **Object** = actual thing created from blueprint

Example:

- `Car` is a class
- `BMW`, `Audi`, `Tesla` are objects

In Python, OOP helps us:

- organize code
- reuse logic
- model real-world entities
- make programs easier to maintain

---

3. Classes

What is a class?

A **class** is a template or blueprint for creating objects.

Example

```
class Student:
  pass
```

This creates a class named `Student`.

VS Code Example: Simple class

Create file `classes_demo.py`

```
class Student:
  pass

s1 = Student()
print(s1)
```

Output

It shows that a `Student` object has been created, something like:

<__main__.Student object at 0x0000021A...>

---

4. Objects

What is an object?

An **object** is an instance of a class.

When you create a class, no real data is stored yet.
When you create an object from that class, memory is allocated.

Example

```
class Student:
  pass

s1 = Student()
```

s2 = Student()

Here:

- s1 is one object
- s2 is another object

Both come from the same class.

Real-life analogy

Think of a class as a building plan.
A constructed house is an object.

---

5. Instance Variables

What are instance variables?

Instance variables are variables that belong to a specific object.

Each object can have its own values.

Example

```
class Student:
  def __init__(self, name, age):
      self.name = name
      self.age = age

s1 = Student("Giri", 25)
s2 = Student("Ravi", 30)

print(s1.name, s1.age)
print(s2.name, s2.age)
```

Output

```
Giri 25
Ravi 30
```

Explanation

- `self.name` and `self.age` are instance variables
- `s1` has its own values
- `s2` has its own values

---

6. Constructors

What is a constructor?

A constructor is a special method automatically called when an object is created.

In Python, the constructor is:

__init__()

Purpose of constructor

It is used to:

- initialize object data
- assign default values
- prepare object state

Example

```
class Employee:
  def __init__(self, emp_id, name, salary):
      self.emp_id = emp_id
      self.name = name
      self.salary = salary

e1 = Employee(101, "Anu", 50000)
print(e1.emp_id)
print(e1.name)
print(e1.salary)
```

Output

101
Anu
50000

Important point

`self` refers to the current object.

So this line:

self.name = name

means:
store the given `name` value inside this object.

Constructor with default values

```
class Product:
  def __init__(self, name="Unknown", price=0):
      self.name = name
      self.price = price

p1 = Product()
p2 = Product("Laptop", 55000)

print(p1.name, p1.price)
print(p2.name, p2.price)
```

Output

Unknown 0
Laptop 55000

---

7. Destructors

What is a destructor?

A destructor is a special method that is called when an object is about to be destroyed.

In Python, it is:

__del__()

Example

```
class Demo:
  def __init__(self, name):
      self.name = name
      print(f"Object {self.name} created")

  def __del__(self):
      print(f"Object {self.name} destroyed")

d1 = Demo("A")
del d1
```

Possible Output

```
Object A created
Object A destroyed
```

Important note about destructors in Python

In Python, destructors are **not used heavily** like in C++ because:

- Python has **garbage collection**
- exact destruction timing is not always predictable

So in real Python programs, destructor usage is less common.

Usually we prefer:

- `with` statement
- explicit cleanup methods
- file closing
- context managers

---

8. Inheritance

What is inheritance?

Inheritance allows one class to reuse properties and methods from another class.

Parent class

Also called:

- base class
- super class

Child class

Also called:

- derived class
- sub class

Example

```
class Animal:
  def eat(self):
      print("Animal is eating")

class Dog(Animal):
  def bark(self):
      print("Dog is barking")

d = Dog()
d.eat()
d.bark()
```

Output

```
Animal is eating
Dog is barking
```

Explanation

`Dog` inherits from `Animal`, so `Dog` gets the `eat()` method.

Practical example

```
class Person:
  def __init__(self, name):
      self.name = name

  def show_name(self):
      print("Name:", self.name)

class Teacher(Person):
  def teach(self):
      print(self.name, "is teaching Python")

t1 = Teacher("Giri")
t1.show_name()
t1.teach()
```

Output

```
Name: Giri
Giri is teaching Python
```

Types of inheritance in Python

Python supports:

- Single inheritance
- Multiple inheritance
- Multilevel inheritance
- Hierarchical inheritance

Single inheritance

```
class A:
  def method_a(self):
      print("Method A")

class B(A):
  def method_b(self):
      print("Method B")
```

Multilevel inheritance

```
class A:
  def method_a(self):
      print("Method A")

class B(A):
  def method_b(self):
      print("Method B")

class C(B):
  def method_c(self):
      print("Method C")

obj = C()
obj.method_a()
obj.method_b()
obj.method_c()
```

Multiple inheritance

```
class Father:
  def skills(self):
      print("Gardening")

class Mother:
  def talents(self):
      print("Painting")

class Child(Father, Mother):
  pass

c = Child()
c.skills()
c.talents()
```

---

9. Polymorphism

What is polymorphism?

Polymorphism means **same method name, different behavior**.

This is one of the core ideas of OOP.

Example 1: Method overriding

```
class Animal:
  def sound(self):
      print("Animal makes sound")

class Dog(Animal):
  def sound(self):
      print("Dog barks")

class Cat(Animal):
  def sound(self):
      print("Cat meows")

a = Animal()
d = Dog()
c = Cat()

a.sound()
d.sound()
c.sound()
```

Output

```
Animal makes sound
Dog barks
Cat meows
```

This is polymorphism because `sound()` behaves differently based on object type.

Example 2: Function working with different objects

```
class Bird:
  def move(self):
      print("Bird flies")

class Fish:
```

```
  def move(self):
      print("Fish swims")

def show_movement(creature):
  creature.move()

show_movement(Bird())
show_movement(Fish())
```

Output

Bird flies
Fish swims

This is also polymorphism.

---

10. Encapsulation

What is encapsulation?

Encapsulation means bundling data and methods together inside a class and controlling access to data.

In Python, encapsulation is usually done using:

- public variables
- protected variables
- private variables

Public variable

```
class Student:
  def __init__(self, name):
      self.name = name

s = Student("Giri")
print(s.name)
```

No restriction.

Protected variable

Conventionally starts with single underscore _

```
class Student:
  def __init__(self, name):
      self._name = name

s = Student("Giri")
print(s._name)
```

It is still accessible, but meant for internal use.

Private variable

Starts with double underscore __

```
class BankAccount:
  def __init__(self, balance):
      self.__balance = balance

  def show_balance(self):
      print("Balance:", self.__balance)

acc = BankAccount(5000)
acc.show_balance()
```

If you try:

```
print(acc.__balance)
```

You get an error.

Example with controlled access

```
class BankAccount:
  def __init__(self, balance):
      self.__balance = balance
```

```
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Invalid withdrawal")

    def get_balance(self):
        return self.__balance

acc = BankAccount(10000)
acc.deposit(2000)
acc.withdraw(3000)
print("Final Balance:", acc.get_balance())
```

Output

Final Balance: 9000

Why encapsulation is useful

- prevents invalid data modification
- improves security
- keeps class logic controlled

---

11. Class Methods and Static Methods

Python has three common method types:

- instance methods
- class methods
- static methods

11.1 Instance method

Works with object data using `self`

```
class Student:
  def __init__(self, name):
      self.name = name

  def display(self):
      print("Student name:", self.name)

s = Student("Giri")
s.display()
```

11.2 Class method

Uses `@classmethod` and takes `cls` instead of `self`

It works with class-level data.

```
class Employee:
  company = "ABC Pvt Ltd"

  def __init__(self, name):
      self.name = name

  @classmethod
  def change_company(cls, new_name):
      cls.company = new_name

e1 = Employee("Anu")
e2 = Employee("Ravi")

print(Employee.company)
Employee.change_company("XYZ Ltd")
print(e1.company)
print(e2.company)
```

Output

```
ABC Pvt Ltd
XYZ Ltd
XYZ Ltd
```

11.3 Static method

Uses `@staticmethod`

It does not depend on:

- object (`self`)
- class (`cls`)

It behaves like a utility function placed inside a class.

```
class MathTools:
  @staticmethod
  def add(a, b):
     return a + b

print(MathTools.add(10, 20))
```

Output

30

Difference summary

Instance method

- uses `self`
- works with object data

Class method

- uses `cls`
- works with class data

Static method

- uses neither `self` nor `cls`
- utility/helper function

---

12. Special Methods

Special methods are also called:

- magic methods
- dunder methods

They begin and end with double underscores.

Examples:

- `__init__`
- `__str__`
- `__len__`
- `__add__`
- `__del__`

These define how objects behave with built-in operations.

Example: `__str__`

```
class Student:
  def __init__(self, name, marks):
      self.name = name
      self.marks = marks

  def __str__(self):
      return f"Student(name={self.name}, marks={self.marks})"

s = Student("Giri", 95)
print(s)
```

Output

Student(name=Giri, marks=95)

Without `__str__`, Python prints object address.

Example: `__len__`

```
class Team:
  def __init__(self, members):
      self.members = members

  def __len__(self):
      return len(self.members)

t = Team(["A", "B", "C"])
print(len(t))
```

Output

3

Example: `__add__`

```
class Number:
  def __init__(self, value):
      self.value = value

  def __add__(self, other):
      return self.value + other.value

n1 = Number(10)
n2 = Number(20)
print(n1 + n2)
```

Output

30

This is operator overloading.

---

13. Iterators

What is an iterator?

An iterator is an object that allows us to loop through data one item at a time.

Examples of iterable objects:

- list
- tuple
- string
- dictionary

Python internally uses iterators in `for` loops.

Example

```
nums = [10, 20, 30]
it = iter(nums)

print(next(it))
print(next(it))
print(next(it))
```

Output

```
10
20
30
```

`iter(nums)` creates an iterator.
`next(it)` gets next value.

What happens after values finish?

If you call `next()` again, Python raises:

StopIteration

Custom iterator example

```
class CountUpTo:
  def __init__(self, limit):
      self.limit = limit
      self.current = 1

  def __iter__(self):
```

```
        return self

    def __next__(self):
        if self.current <= self.limit:
            value = self.current
            self.current += 1
            return value
        else:
            raise StopIteration

counter = CountUpTo(5)
for num in counter:
    print(num)
```

Output

```
1
2
3
4
5
```

---

14. Generators

What is a generator?

A generator is a simpler way to create iterators.

Instead of writing `__iter__()` and `__next__()`, we use `yield`.

Example

```
def count_up_to(limit):
    num = 1
    while num <= limit:
        yield num
        num += 1

for value in count_up_to(5):
    print(value)
```

Output

```
1
2
3
4
5
```

Why generators are useful

- memory efficient
- good for large data
- values produced one by one
- easier than custom iterator classes

Generator vs normal function

Normal function

```
def normal():
  return [1, 2, 3]
```

Generator

```
def gen():
  yield 1
  yield 2
  yield 3
```

Normal function returns all values at once.
Generator yields one value at a time.

Example: reading large range

```
def square_generator(n):
  for i in range(1, n + 1):
      yield i * i

for sq in square_generator(5):
  print(sq)
```

Output

```
1
4
9
16
25
```

---

15. Decorators

What is a decorator?

A decorator is a function that modifies or extends another function without changing its original code.

This is a very important intermediate Python feature.

Basic idea

A decorator wraps another function.

Example

```
def my_decorator(func):
  def wrapper():
      print("Before function execution")
      func()
      print("After function execution")
  return wrapper

@my_decorator
def say_hello():
  print("Hello")

say_hello()
```

Output

```
Before function execution
Hello
After function execution
```

Without @ syntax

The above is equal to:

```
def say_hello():
  print("Hello")

say_hello = my_decorator(say_hello)
say_hello()
```

Practical example: logging decorator

```
def log_function(func):
  def wrapper(*args, **kwargs):
      print(f"Calling function: {func.__name__}")
      result = func(*args, **kwargs)
      print(f"Function {func.__name__} finished")
      return result
  return wrapper

@log_function
def add(a, b):
  return a + b

print(add(5, 3))
```

Output

```
Calling function: add
Function add finished
8
```

Practical use cases of decorators

- logging
- authentication

- authorization
- timing functions
- caching
- validation

---

16. Higher-Order Functions

What is a higher-order function?

A higher-order function is a function that:

- takes another function as argument, or
- returns a function

Example 1: function passed as argument

```
def greet(name):
  return f"Hello {name}"

def process_user(func, name):
  return func(name)

print(process_user(greet, "Giri"))
```

Output

Hello Giri

Example 2: function returned from another function

```
def outer_function():
  def inner_function():
      print("Inside inner function")
  return inner_function

result = outer_function()
result()
```

Output

Inside inner function

Decorators are based on higher-order functions.

---

17. map, filter, reduce

These are functional programming tools in Python.

17.1 map()

`map()` applies a function to each item in an iterable.

Example

```
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x * x, numbers))
print(squares)
```

Output

[1, 4, 9, 16, 25]

Another example

```
names = ["giri", "ravi", "anu"]
upper_names = list(map(str.upper, names))
print(upper_names)
```

Output

['GIRI', 'RAVI', 'ANU']

17.2 filter()

`filter()` selects items based on a condition.

Example

```
numbers = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)
```

Output

```
[2, 4, 6]
```

17.3 reduce()

`reduce()` repeatedly applies a function to combine values into one result.

It must be imported from `functools`.

Example

```
from functools import reduce

numbers = [1, 2, 3, 4]
result = reduce(lambda a, b: a + b, numbers)
print(result)
```

Output

```
10
```

More example

```
from functools import reduce

numbers = [1, 2, 3, 4]
product = reduce(lambda a, b: a * b, numbers)
print(product)
```

Output

```
24
```

---

map/filter/reduce vs loops

Many times, beginners understand loops more easily.

With loop

```
numbers = [1, 2, 3, 4]
squares = []

for n in numbers:
  squares.append(n * n)

print(squares)
```

With map

```
numbers = [1, 2, 3, 4]
squares = list(map(lambda n: n * n, numbers))
print(squares)
```

Both are valid.

In Python, list comprehensions are often preferred for readability.

---

18. JSON Handling

What is JSON?

JSON stands for:
JavaScript Object Notation

It is a common format for storing and exchanging data.

Python has built-in `json` module.

Python object to JSON

Use:

json.dumps()

Example

```
import json
```

```
student = {
   "name": "Giri",
   "age": 25,
   "skills": ["Python", "SQL", "Git"]
}

json_data = json.dumps(student)
print(json_data)
print(type(json_data))
```

Output

```
{"name": "Giri", "age": 25, "skills": ["Python", "SQL", "Git"]}
<class 'str'>
```

JSON to Python object

Use:

```
json.loads()
```

Example

```
import json

json_string = '{"name": "Ravi", "age": 30}'
data = json.loads(json_string)

print(data)
print(type(data))
print(data["name"])
```

Output

```
{'name': 'Ravi', 'age': 30}
<class 'dict'>
Ravi
```

Writing JSON to a file

```
import json
```

```
data = {
  "id": 101,
  "name": "Laptop",
  "price": 55000
}

with open("product.json", "w") as file:
  json.dump(data, file, indent=4)

print("JSON written to file")
```

This creates `product.json`.

Reading JSON from a file

```
import json

with open("product.json", "r") as file:
  data = json.load(file)

print(data)
print(data["name"])
```

Real-world use of JSON

JSON is heavily used in:

- web APIs
- configuration files
- data exchange
- frontend-backend communication

---

19. Basic Regex

What is regex?

Regex means **regular expression**.

It is used to:

- search patterns
- validate text
- extract data
- replace text

Python uses `re` module.

Example 1: simple search

```
import re

text = "My phone number is 9876543210"
match = re.search(r"\d+", text)

if match:
  print("Found:", match.group())
```

Output

Found: 9876543210

Example 2: find all numbers

```
import re

text = "Marks: 85, 90, 78"
numbers = re.findall(r"\d+", text)
print(numbers)
```

Output

['85', '90', '78']

Example 3: validate email pattern

```
import re

email = "test123@gmail.com"
pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
```

```
if re.match(pattern, email):
  print("Valid email")
else:
  print("Invalid email")
```

Example 4: replace text

```
import re

text = "Python is easy. Python is powerful."
result = re.sub(r"Python", "Java", text)
print(result)
```

Output

Java is easy. Java is powerful.

Common regex symbols

`\d`

Digit

Example:

r"\d"

Matches one digit.

`\w`

Word character

`\s`

Whitespace

`+`

One or more

`*`

Zero or more

`^`

Start of string

`$`

End of string

`[abc]`

Match a, b, or c

`[0-9]`

Any digit from 0 to 9

Example: extract emails from text

```
import re

text = "Contact us at abc@test.com and hr@company.org"
emails = re.findall(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}", text)
print(emails)
```

Output

['abc@test.com', 'hr@company.org']

---

20. Combined practical example

This example mixes:

- class
- constructor
- instance variable
- special method
- JSON
- regex

```
import json
import re

class User:
    def __init__(self, name, email):
        self.name = name
        self.email = email

    def is_valid_email(self):
        pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
        return re.match(pattern, self.email) is not None

    def to_dict(self):
        return {
            "name": self.name,
            "email": self.email
        }

    def __str__(self):
        return f"User(name={self.name}, email={self.email})"

u1 = User("Giri", "giri@example.com")
print(u1)

if u1.is_valid_email():
    json_data = json.dumps(u1.to_dict(), indent=4)
    print(json_data)
else:
```

```
    print("Invalid email")
```

---

21. Full practice file for VS Code

Create `main.py` and paste this:

```
from functools import reduce
import json
import re

print("---- OOP DEMO ----")

class Person:
    country = "India"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")

    @classmethod
    def change_country(cls, new_country):
        cls.country = new_country

    @staticmethod
    def is_adult(age):
        return age >= 18

    def __str__(self):
        return f"Person(name={self.name}, age={self.age})"


class Employee(Person):
    def __init__(self, name, age, emp_id):
        super().__init__(name, age)
        self.emp_id = emp_id

    def display(self):
        print(f"Employee ID: {self.emp_id}, Name: {self.name}, Age: {self.age}")
```

```
p1 = Person("Giri", 25)
p1.display()
print(p1)
print("Adult?", Person.is_adult(25))

e1 = Employee("Ravi", 30, 101)
e1.display()

print("\n---- ITERATOR / GENERATOR ----")

nums = [10, 20, 30]
it = iter(nums)
print(next(it))
print(next(it))
print(next(it))

def simple_generator():
  yield 1
  yield 2
  yield 3

for value in simple_generator():
  print("Generated:", value)

print("\n---- DECORATOR ----")

def log_call(func):
  def wrapper(*args, **kwargs):
      print(f"Calling {func.__name__}")
      result = func(*args, **kwargs)
      print(f"Finished {func.__name__}")
      return result
  return wrapper

@log_call
def add(a, b):
  return a + b

print(add(5, 7))

print("\n---- MAP FILTER REDUCE ----")

numbers = [1, 2, 3, 4, 5]
```

```
squares = list(map(lambda x: x * x, numbers))
evens = list(filter(lambda x: x % 2 == 0, numbers))
total = reduce(lambda a, b: a + b, numbers)

print("Squares:", squares)
print("Evens:", evens)
print("Total:", total)

print("\n---- JSON ----")

student = {
   "name": "Anu",
   "age": 22,
   "course": "Python"
}

json_text = json.dumps(student, indent=4)
print(json_text)

parsed = json.loads(json_text)
print("Parsed Name:", parsed["name"])

print("\n---- REGEX ----")

text = "Contact: test@gmail.com, phone: 9876543210"
email_match = re.findall(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}", text)
number_match = re.findall(r"\d+", text)

print("Emails:", email_match)
print("Numbers:", number_match)
```

Run in terminal:

```
python main.py
```

---

22. Beginner understanding of all requested topics in one place

OOP topics

- **Class** → blueprint
- **Object** → instance created from class
- **Instance variable** → object-specific data
- **Constructor** → initializes object
- **Destructor** → cleanup before object deletion
- **Inheritance** → child class reuses parent features
- **Polymorphism** → same method name, different behavior
- **Encapsulation** → restrict direct access and protect data
- **Class method** → works with class data
- **Static method** → utility method
- **Special methods** → built-in behavior customization

Intermediate topics

- **Iterator** → object used for step-by-step traversal
- **Generator** → easier iterator using `yield`
- **Decorator** → wraps function to extend behavior
- **Higher-order function** → function taking/returning functions
- **map/filter/reduce** → functional processing tools
- **JSON handling** → convert between Python and JSON
- **Regex** → pattern matching and validation

---

23. Common mistakes beginners make

1. Forgetting `self`

Wrong:

```
class Student:
  def __init__(name, age):
    self.name = name
```

Correct:

```
class Student:
  def __init__(self, name, age):
    self.name = name
```

---

2. Confusing class variable and instance variable

```
class Demo:
  x = 10   # class variable

  def __init__(self):
     self.y = 20   # instance variable
```

---

3. Using decorator without wrapper parameters

If decorated function has arguments, wrapper should usually use:

```
def wrapper(*args, **kwargs):
```

---

4. Forgetting to convert map/filter result to list

In Python 3, `map()` and `filter()` return iterator objects.

So:

```
result = map(lambda x: x*x, [1,2,3])
print(list(result))
```

---

5. Wrong regex escaping

Prefer raw string:

```
r"\d+"
```

instead of:

```
"\\d+"
```

---

24. Tips

In Python, OOP is important, but Python is also a very flexible language. Good Python developers do not force everything into classes unnecessarily.

Use:

- **classes** when modeling entities and behavior
- **functions** when a simple functional approach is enough
- **generators** for memory-efficient processing
- **decorators** for reusable cross-cutting behavior
- **JSON + regex** heavily in API, automation, scripting, and data processing work

A strong intermediate Python learner should be comfortable combining these topics together in one program.

Example:

- Read JSON data from file
- Convert records into class objects
- Validate using regex
- Process using `map/filter`
- Generate output using a generator
- Add logging using decorators

That is where Python becomes really useful in real projects.

---

25. Key points

Practice these in this order inside VS Code:

1. Classes, objects, constructor
2. Inheritance, polymorphism, encapsulation
3. Class/static methods and special methods
4. Iterators and generators
5. Decorators and higher-order functions
6. map/filter/reduce
7. JSON handling
8. Regex